Global Capacity Orchestrator (GCO) — Demo Walkthrough

One API. Every Accelerator. Any Region.

Prepared by [Your Name] · [Your Title] · [Your Team]

Table of Contents

- What You're About to See
- Prerequisites
- Demo Segment 1: Platform Configuration
- Demo Segment 2: Capacity Discovery and Job Submission
- Demo Segment 3: Autoscaling in Action
- Demo Segment 4: Persistent Storage
- Demo Segment 5: Inference Endpoints
- Demo Segment 6: Bonus Commands
- Demo Segment 7: AWS Trainium and Inferentia
- Quick Reference
- Architecture Overview
- Key Technical Details
- Links

What You're About to See

Global Capacity Orchestrator (GCO) is a production-ready platform that submits GPU workloads across any number of SDK-known AWS Regions in one partition. This commercial `aws` demo uses EKS Auto Mode, Global Accelerator, and API Gateway; other partitions omit Global Accelerator and use IAM-authenticated regional API bridges for workload traffic.

This document walks through the live demo so you can follow along or replicate it in your own account afterward.

Repository: https://github.com/awslabs/global-capacity-orchestrator-on-aws

---

Prerequisites

To replicate this demo, you'll need:

- An AWS account with GPU quota in at least one region
- AWS CLI configured with appropriate credentials
- Python 3.14+, Node.js (LTS), CDK CLI
- A container runtime (Docker, Podman or Finch)

```
aws sts get-caller-identity
python3 --version
cdk --version
```

Install the GCO CLI:

```
git clone https://github.com/awslabs/global-capacity-orchestrator-on-
        aws.git
cd global-capacity-orchestrator-on-aws
pipx install -e .
gco --version
```

Demo Segment 1: Platform Configuration (~1.5 min)

The entire platform is defined in a single configuration file. Adding a region means adding one line.

Show the config:

```
cat cdk.json | python3 -m json.tool
```

Key things to notice:

- `deployment_regions.regional` — list of regions where EKS clusters are deployed
- `deployment_regions.global` — where partition-wide state lives (and Global Accelerator in commercial `aws`)
- `eks_cluster.endpoint_access: PRIVATE` — clusters are private by default

- `job_validation_policy.trusted_registries` — only approved container registries are allowed
- `fsx_lustre.enabled` — toggle for high-performance storage
- `valkey.enabled` — toggle for Valkey Serverless cache

Show deployed stacks:

```
gco stacks list
gco stacks status gco-us-east-1 --region us-east-1
```

Note: Full deployment takes under 60 minutes via `gco stacks deploy-all -y`. For the demo, infrastructure is pre-deployed.

Important: By default, EKS clusters are deployed with `endpoint_access: PRIVATE`, meaning kubectl can only reach the API server from within the VPC. For the demo, we temporarily switch to `PUBLIC_AND_PRIVATE` so we can run kubectl from a local machine. To do this before the demo:

```
python3 -c "
import json, pathlib
p = pathlib.Path('cdk.json')
c = json.loads(p.read_text())
c['context']['eks_cluster']['endpoint_access'] = 'PUBLIC_AND_PRIVATE'
p.write_text(json.dumps(c, indent=2) + '\n')
"
```

Then redeploy (takes ~5-10 minutes for the endpoint change):

```
gco stacks deploy gco-us-east-1 -y
```

In production, keep `PRIVATE` and submit jobs via SQS or API Gateway — no kubectl or cluster access needed.

Demo Segment 2: Capacity Discovery and Job Submission (~3.5 min)

This is the core value proposition — finding GPU capacity and submitting work without managing clusters.

Check GPU availability:

```
gco capacity check --instance-type g5.xlarge --region us-east-1
gco capacity recommend-region --gpu
```

The CLI checks Spot Placement Scores and instance availability across regions to find where GPUs are actually available right now.

Look at a GPU job manifest:

```
cat examples/gpu-job.yaml
```

This is a standard Kubernetes Job that requests `nvidia.com/gpu: 1`. Nothing GCO-specific in the manifest — it's portable.

Submit with automatic region selection:

```
gco jobs submit-sqs examples/gpu-job.yaml --auto-region
```

What just happened: the CLI analyzed capacity across all configured regions, selected the best one, and placed the job manifest on that region's SQS queue. The queue processor — a KEDA ScaledJob running in each cluster — automatically picks up the message and applies the manifest to Kubernetes. No manual intervention needed.

Verify it was queued and picked up:

```
gco jobs queue-status --all-regions
gco jobs list --region us-east-1
```

Alternative submission methods:

GCO supports four ways to submit jobs, depending on your use case:

1. SQS queue (recommended) — async, auto-processed by the queue processor:

   ```
   gco jobs submit-sqs examples/gpu-job.yaml --region us-east-1
   ```

2. API Gateway — synchronous, SigV4 authenticated:

   ```
   gco jobs submit examples/gpu-job.yaml -n gco-jobs
   ```

3. Direct kubectl — requires an EKS access entry:

```
gco jobs submit-direct examples/gpu-job.yaml -r us-east-1
```

4. DynamoDB global queue — centralized tracking with status history:

```
gco jobs submit-queue examples/gpu-job.yaml --region us-east-1
```

Demo Segment 3: Autoscaling in Action (~4 min)

This is where EKS Auto Mode shines. GPU nodes are provisioned on-demand and removed when idle.

In a second terminal, set up cluster access and watch nodes:

```
./scripts/setup-cluster-access.sh gco-us-east-1 us-east-1
kubectl get nodes -w
```

Submit a GPU job:

```
gco jobs submit examples/gpu-job.yaml
```

Watch the pod get scheduled:

```
kubectl get pods -n gco-jobs -w
```

What you'll see: EKS Auto Mode detects the pending GPU pod, provisions a g5.xlarge instance (~60-90 seconds), the pod runs `nvidia-smi`, and completes.

View the job output (run while the pod is still alive or recently completed):

```
gco jobs logs gpu-test-job -n gco-jobs -r us-east-1
```

You should see the `nvidia-smi` output showing the GPU details.

Watch scale-to-zero:

Wait ~30 seconds after the job completes, then:

```
kubectl get nodes
```

The GPU node is gone. You only pay for GPU compute while jobs are actually running.

Demo Segment 4: Persistent Storage (~2 min)

When Kubernetes pods terminate, their local data disappears. GCO solves this with shared EFS storage.

Look at the EFS output job:

```
cat examples/efs-output-job.yaml
```

This job writes results to `/outputs` which is backed by an EFS PersistentVolumeClaim.

Submit and wait for completion:

```
gco jobs submit examples/efs-output-job.yaml --wait
```

Download the results — even after the pod is gone:

```
gco files ls -r us-east-1
gco files download efs-output-example ./demo-results -r us-east-1
cat demo-results/results.json
```

The pod terminated, but the data persists on EFS. This is critical for ML training checkpoints and inference results that need to survive job completion.

Demo Segment 5: Inference Endpoints (~3 min)

GCO isn't just for batch jobs — it also manages multi-region inference endpoints with a single command.

Deploy a vLLM endpoint:

```
gco inference deploy vllm-demo \
  -i vllm/vllm-openai:v0.24.0 \
  --gpu-count 1 \
  --replicas 1 \
  --extra-args '--model' --extra-args 'facebook/opt-125m'
```

Omitting `-r` deploys to every configured Region, keeping endpoint availability consistent. In this commercial `aws` demo that also makes Global Accelerator routing safe. The inference_monitor in each Region picks up the DynamoDB record and creates the Kubernetes Deployment and internal ClusterIP Service. The existing shared Ingress routes `/inference/*` through the dedicated authenticated inference proxy before it reaches that Service; no endpoint-specific Ingress is created.

Check status and invoke:

```
gco inference status vllm-demo

gco inference invoke vllm-demo \
  -p "Explain GPU orchestration for ML workloads."

# Optional: print chunks as the model emits them
gco inference invoke vllm-demo \
  -p "Explain GPU orchestration for ML workloads." --stream
```

Scale and clean up:

```
gco inference scale vllm-demo --replicas 3
gco inference status vllm-demo
gco inference delete vllm-demo -y
```

See the Inference Walkthrough for the full inference demo including canary deployments, autoscaling, model weight management, and more.

Demo Segment 6: Bonus Commands (~2 min)

AI-powered capacity recommendations (Amazon Bedrock):

```
gco capacity ai-recommend \
  --workload "Training a large language model" \
  --gpu \
  --min-gpus 4
```

Job templates for reusable workflows:

```
gco templates create examples/gpu-job.yaml \
  --name gpu-training \
  -d "GPU training template"
gco templates list
gco templates run gpu-training --name my-run --region us-east-1
```

DAG pipelines for multi-step workflows:

```
gco dag run examples/pipeline-dag.yaml --dry-run
gco dag run examples/pipeline-dag.yaml -r us-east-1
```

Cost tracking:

```
gco costs summary
gco costs regions
gco costs trend --days 7
gco costs workloads
```

Global job queue (DynamoDB-backed):

```
gco queue list
gco queue stats
```

Demo Segment 7: AWS Trainium and Inferentia (~2 min)

GCO includes built-in support for AWS Trainium and Inferentia accelerators. These are purpose-built ML chips that use the Neuron SDK instead of CUDA, offering lower cost for training and inference workloads.

Submit a job on Inferentia (cost-optimized inference):

```
gco jobs submit examples/inferentia-job.yaml --region us-east-1
gco jobs get inferentia-test -r us-east-1
gco jobs logs inferentia-test -r us-east-1
```

Submit a job on Trainium (cost-optimized training):

```
gco jobs submit examples/trainium-job.yaml --region us-east-1
gco jobs get trainium-test -r us-east-1
gco jobs logs trainium-test -r us-east-1
```

Deploy inference on Neuron accelerators:

```
gco inference deploy my-model \
  -i public.ecr.aws/neuron/your-neuron-image:latest \
  --gpu-count 1 --accelerator neuron
```

Talking points:

- Karpenter automatically provisions the right instance type (inf2 for Inferentia, trn1/trn2 for Trainium) based on the node affinity in the manifest
- The `--accelerator neuron` flag on `gco inference deploy` sets up the correct resource requests (`aws.amazon.com/neuron`), tolerations, and node selectors
- A dedicated Neuron nodepool with taints prevents non-Neuron workloads from accidentally scheduling on these instances
- The Neuron device plugin is installed automatically via Helm chart

---

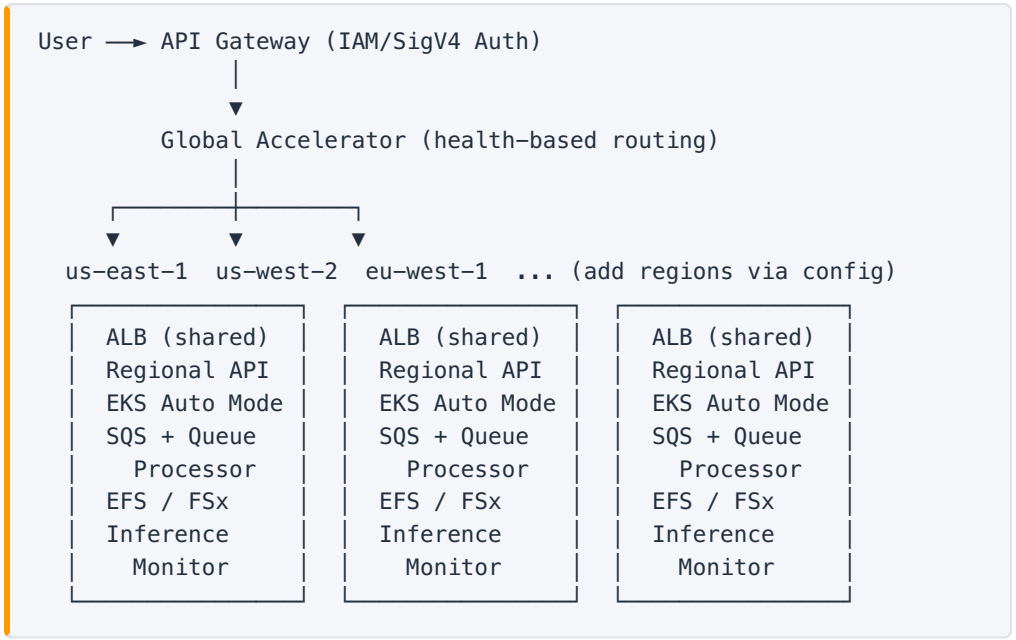
Quick Reference

Action	Command
Deploy all infrastructure	`gco stacks deploy-all -y`
Configure kubectl access	`gco stacks access -r us-east-1`
Destroy all infrastructure	`gco stacks destroy-all -y`
Check GPU capacity	`gco capacity check -i g5.xlarge -r us-east-1`
Find best region for GPUs	`gco capacity recommend-region --gpu`
AI capacity recommendation	`gco capacity ai-recommend --workload "description" --gpu`
Submit job (SQS, auto-region)	`gco jobs submit-sqs job.yaml --auto-region`
Submit job (SQS, specific region)	`gco jobs submit-sqs job.yaml -r us-east-1`
Submit job (API Gateway)	`gco jobs submit job.yaml -n gco-jobs`
Submit job (direct kubectl)	`gco jobs submit-direct job.yaml -r us-east-1`
Submit job (DynamoDB queue)	`gco jobs submit-queue job.yaml -r us-east-1`
List jobs	`gco jobs list -r us-east-1`
List jobs (all regions)	`gco jobs list --all-regions`
View job logs	`gco jobs logs JOB_NAME -n gco-jobs -r us-east-1`
Wait for job completion	`gco jobs submit job.yaml --wait`
Check queue depth (SQS)	`gco jobs queue-status --all-regions`
Check queue depth (DynamoDB)	`gco queue stats`
Cluster health	`gco jobs health --all-regions`
List files on shared storage	`gco files ls -r us-east-1`
Download job outputs	`gco files download PATH ./local -r us-east-1`

Action	Command
Deploy inference endpoint	`gco inference deploy NAME -i IMAGE --gpu-count 1`
Deploy inference on Neuron	`gco inference deploy NAME -i IMAGE --gpu-count 1 --accelerator neuron`
Invoke inference endpoint	`gco inference invoke NAME -p "prompt"`
Scale inference endpoint	`gco inference scale NAME --replicas 3`
List nodepools	`gco nodepools list -r us-east-1`
Run DAG pipeline	`gco dag run pipeline.yaml -r us-east-1`
Cost summary	`gco costs summary`
Create job template	`gco templates create job.yaml -n NAME`

Architecture Overview

The recorded demo uses the commercial `aws` topology:



Other AWS partitions omit Global Accelerator; users call the selected Region's IAM-authenticated regional API Gateway, whose VPC Lambda reaches the same internal ALB. Each Region runs an independent EKS Auto Mode cluster with GPU nodepools, shared storage, health monitoring, SQS job queues with automatic processing, inference endpoint reconciliation, and manifest processing. Adding an SDK-known Region in the deployment's partition is a one-line config change and a redeploy.

Key Technical Details

- **Authentication:** IAM-native (SigV4) — uses your existing AWS credentials, no kubeconfig management
- **Compute:** EKS Auto Mode provisions GPU nodes on-demand (g4dn, g5, g5g, p4d, p5) and scales to zero when idle
- **Storage:** EFS for general outputs, FSx for Lustre for high-throughput ML training (optional), Valkey Serverless for caching (optional)
- **Networking:** In commercial `aws`, Global Accelerator provides a single workload endpoint with automatic failover; other partitions use IAM-authenticated regional workload endpoints
- **Job Submission:** Four paths — SQS queue with auto-processing (recommended), API Gateway, direct kubectl, or DynamoDB global queue
- **Queue Processing:** KEDA ScaledJob automatically consumes SQS messages and applies manifests to the cluster — no manual intervention

- **Inference Serving:** DynamoDB-backed desired state with continuous reconciliation, canary deployments, autoscaling, and model weight sync from S3
- **Compliance:** CDK-nag validated against HIPAA, NIST 800-53, PCI DSS
- **Cost Tracking:** Built-in cost breakdown by service, region, and workload with forecasting
- **Pipelines:** DAG-based multi-step workflows with dependency ordering

---

Links

- **Repository:** https://github.com/awslabs/global-capacity-orchestrator-on-aws
- **Architecture:** https://github.com/awslabs/global-capacity-orchestrator-on-aws/blob/main/docs/ARCHITECTURE.md
- **CLI Reference:** https://github.com/awslabs/global-capacity-orchestrator-on-aws/blob/main/docs/CLI.md
- **Quick Start:** https://github.com/awslabs/global-capacity-orchestrator-on-aws/blob/main/QUICKSTART.md
- **Examples:** https://github.com/awslabs/global-capacity-orchestrator-on-aws/tree/main/examples

---

Questions? Check the repository for full documentation.